

LEON3 Ethernet test code documentation and execution guide ▾

greth_test.c

C

i have uploaded a c file for a etherent test for leon3 processor that runs in vcs on a linux terminal...can u me a document that explains what the c code is doing and how to run it and see it's working

Viewed 2 files >

Now let me read the docx skill and create the document.

Reading docx skill >

Now I have everything I need. Let me write the document generation script.

Ran 3 commands, created a file, read a file >

Here's your Word document! It covers everything in 9 sections:

1. **Overview** — what the program is and how it self-checks
2. **Hardware & Memory Map** — all register addresses, the GRLIB report mechanism, and DMA descriptor layout
3. **Code Structure & Function Reference** — a table of every function and what it does
4. **Test-by-Test Explanation** — detailed breakdown of all 5 tests including exact step sequences for Tests 4 & 5
5. **How to Build and Run** — prerequisites, the 4 steps (copy → compile → vcs-launch → read transcript), and common make targets
6. **Expected VCS Transcript Output** — the exact console output you should see when everything passes (22 checks)
7. **Troubleshooting** — 7 common failure symptoms with causes and fixes
8. **Key Configurable Constants** — FRAME_SIZE, NUM_FRAMES, TIMEOUT, PHY_ADDR
9. **Pass/Fail Summary** — what each test's PASS actually proves at the hardware level



Greth test documentation



this test will work even if the ethernet is taken outside the leon3 processor ryt? or will that be a different testing code?

Great question. The short answer is: **this specific code will NOT work outside the LEON3 processor** — you would need a different testing approach. Here's why:

Why this code is LEON3-specific

1. Bare-metal memory-mapped register access The entire test works by casting hardcoded addresses like `0x80000D00` to pointers and reading/writing them directly. This only works when your code is *running on the LEON3 itself*, where the CPU's memory bus is physically connected to those APB/AHB registers.

2. The addresses are from LEON3's memory map `0x80000D00` (GRETH), `0x80000200` (GRLIB testbench report) — these are LEON3 system addresses defined in `config.vhd`. A CPU outside the SoC has no visibility into these addresses at all.

3. The DMA descriptors live in LEON3's SRAM Tests 4 & 5 write descriptor structs into LEON3 local memory and pass their addresses to GRETH's AHB DMA. An external tester can't do this — it has no access to that memory space.

4. The compiler target is SPARC `sparc-gaisler-elf-gcc` produces SPARC V8 machine code. It won't run on an x86 Linux host or any external microcontroller.

What "external" testing would look like

It depends on *what* is external:

Scenario	Approach
Test from a Linux PC over real Ethernet	Send/receive raw Ethernet frames using <code>libpcap</code> or raw sockets (<code>AF_PACKET</code>) on the PC side; the LEON3 runs a simple loopback firmware
Test from another FPGA/board	A second device sends known frames; you capture and verify them on the receiving end
Test the GRETH IP core in isolation (without LEON3)	Write a pure VHDL/SystemVerilog testbench that drives the AHB/APB buses directly — no CPU involved at all

Scenario	Approach
Use GRETH's built-in EDCL debug link	GRETH has an Ethernet Debug Communication Link (EDCL) that lets a host PC talk to LEON3 memory over UDP — but this still requires LEON3 to be running

Bottom line

This code is tightly coupled to:

- The **LEON3 CPU** executing it
- The **GRLIB SoC memory map**
- The **VCS + GRLIB testbench** infrastructure

If you want to test GRETH from *outside* the processor, you'd need either a **VHDL-level testbench** (drives the bus signals directly) or a **host-side packet tool** (libpcap/raw sockets) paired with minimal firmware still running on LEON3. Want me to sketch out what either of those would look like?

i mean to ask that if i took my ethernet module outside the chip and then interfaced it again to the leon3 processor..will it work

Ah, got it! You're asking about a **custom external Ethernet MAC/PHY** interfaced back to LEON3, replacing the built-in GRETH core. In that case:

Short Answer: No, this code won't work as-is — but it's fixable.

Why it breaks

The test code is hardwired to talk to **GRETH's specific register set**. If you replace GRETH with your own external Ethernet module, your module will have:

- A **different base address** (wherever you map it on the APB/AHB bus)
- A **different register layout** (your own CTRL, STATUS, TX/RX descriptor format)
- Possibly a **different bus interface** (AHB, APB, SPI, parallel — depending on how you connect it)
- A **different descriptor/DMA mechanism** (or no DMA at all — maybe you use FIFO-based transfer)

So the test code would be writing to wrong addresses and expecting register bits that don't exist in your module.

What stays the same vs what changes

Part of the code	Stays the same?	Why
<code>printf</code> / <code>check()</code> / <code>delay()</code>	✔ Yes	Pure software, CPU-independent
<code>REPORT_START</code> / <code>REPORT_END</code> macros	✔ Yes	GRLIB testbench mechanism, not GRETH-specific
<code>sparc-gaisler-elf-gcc</code> compilation	✔ Yes	Still running on LEON3
<code>make vcs-launch</code> flow	✔ Yes	Simulation flow unchanged
<code>GRETH_BASE</code> and all register addresses	✗ No	Your module lives at a different address
<code>GRETH_CTRL/STATUS/MDIO</code> register bits	✗ No	Your register map will differ
DMA descriptor structure (<code>desc_t</code>)	✗ Probably not	Depends on your module's DMA design
MDIO read/write functions	⚠ Maybe	Only if your module still uses standard MDIO for PHY access
PHY loopback via MDIO	⚠ Maybe	Works if you kept a standard MII PHY

What you'd need to rewrite

Essentially you'd replace the **hardware abstraction layer** — the register definitions and low-level driver functions — while keeping the test logic and pass/fail structure:

```
c
// Change this block to match YOUR module's memory map
#define MY_ETH_BASE      0x80000X00    // wherever you mapped it

#define MY_ETH_CTRL      (MY_ETH_BASE + 0x00)
#define MY_ETH_STATUS    (MY_ETH_BASE + 0x04)
// ... your register layout here
```

```
// Rewrite mdio_read/write if your PHY interface changed
// Rewrite descriptor setup if your DMA format changed
```

The test structure (5 tests, check(), pass/fail summary) can stay exactly the same — you'd just be poking your registers instead of GRETH's.

The key question to ask yourself

How did you interface your external Ethernet module back to LEON3?

- If via **APB** → relatively easy remap, same coding style
- If via **AHB** → similar, but descriptor/DMA handling may change
- If via **SPI or parallel GPIO** → bigger rewrite; you'll need a software driver loop instead of DMA descriptors
- If you kept a **standard MII/RMII PHY** → the MDIO functions are reusable as-is

Want me to help you write the adapted test code once you know your module's register map and bus interface?

i won't be replacing the GRETH with my own ethernet..I will be taking the GRETH outside and convert it's apb and ahb controls into memory interface and then use a ahb/apb to memory interface bridge convertor to access the apb and ahb signals inside the check and connect them to the GRETH outside(now has memory interface) via the bridge..will this test work for that?

Ah now I understand completely! So your architecture is:

```
LEON3 SoC
├─ AHB/APB bus (internal)
├─ AHB→Memory bridge (inside chip)
└─ Memory interface pins → [off-chip] → GRETH (external, memory-mapped)
```

Short Answer: Yes, this test code will work — with no changes (ideally) or very minor ones.

Why it still works

The test code **doesn't care** where GRETH physically lives. It only does this:

c

```
WR(0x80000D00, value); // writes to an address
RD(0x80000D00);        // reads from an address
```

As long as your AHB/APB → Memory bridge correctly maps address `0x80000D00` to the external GRETH's corresponding register, LEON3 has no idea GRETH is now sitting outside the chip. The memory-mapped abstraction is transparent to the CPU. This is exactly what memory-mapped I/O bridges are designed for.

What the bridge must guarantee

For the test to pass without any code changes, your bridge must preserve:

Requirement	Why the test depends on it
Address mapping is 1:1 — <code>0x80000D00</code> still reaches GRETH CTRL	Every <code>RD()</code> / <code>WR()</code> in the test uses hardcoded addresses
Read/write semantics are preserved — no extra latency that drops data	<code>mdio_wait()</code> polls in a tight loop; stale reads will cause false timeouts
Descriptor physical addresses are still valid — <code>txd[]</code> and <code>rx[]</code> addresses that GRETH's AHB DMA uses must be reachable	Tests 4 & 5 pass <code>(unsigned int)txd</code> directly to <code>GRETH_TXDESC</code>
AHB DMA path is preserved — GRETH's DMA must still reach LEON3's SRAM	GRETH fetches frame data from <code>txbuf[]</code> in LEON3 memory over AHB

The one thing that could cause issues

The DMA path is the tricky part. Look at Test 4:

c

```
txd[0].addr = (unsigned int)txbuf[0]; // address in LEON3 SRAM
WR(GRETH_TXDESC, (unsigned int)txd); // GRETH will DMA-fetch from this address
```

GRETH (now external) needs to DMA back into LEON3's SRAM over AHB to fetch the frame data. So your bridge needs to handle both directions:

LEON3 CPU → bridge → GRETH registers (register access, APB)
GRETH DMA → bridge → LEON3 SRAM (data fetch, AHB – reverse direction!)

If your bridge only handles CPU→GRETH direction and **not** the reverse DMA path, Tests 4 & 5 will fail (TX/RX never complete) while Tests 1, 2, 3 will pass fine.

Summary

Test	Will it pass?
Test 1 — Register access	✔ Yes — pure APB reads/writes through bridge
Test 2 — MDIO read	✔ Yes — APB only
Test 3 — PHY loopback	✔ Yes — APB only
Test 4 — Single frame loopback	✔ Only if bidirectional DMA path works
Test 5 — Multi-frame stress	✔ Only if bidirectional DMA path works

So essentially this test is a **perfect validation tool for your bridge design** — if all 5 tests pass, your bridge is correctly handling both APB register access and AHB DMA in both directions. If Tests 1–3 pass but 4–5 fail, you know exactly where the bridge has a gap.

ok...can i try running this on vc code now? will it give any output or no?

No, you cannot run this on VS Code and get meaningful output. Here's why:

Why it won't work on VS Code / your PC

This is **not a regular C program**. It is **bare-metal embedded firmware** written for a specific hardware target. Three fundamental blockers:

1. The addresses don't exist on your PC

c

WR(0x80000D00, value) // writes to LEON3's APB bus address
RD(0x80000200) // GRLIB testbench report address

On your PC, address `0x80000D00` is either unmapped memory or belongs to some random OS process. Writing to it will cause an **immediate segfault / access violation** and the program will crash on the very first register access.

2. The compiler target is wrong

The code is meant to be compiled with:

```
sparc-gaisler-elf-gcc ← SPARC architecture cross-compiler
```

If you compile it with a normal PC compiler (gcc/clang/MSVC), you get an x86 or ARM binary — wrong instruction set entirely, and none of the hardware assumptions hold.

3. There is no GRETH, no PHY, no AHB bus on your PC

Even if you bypassed the address crash, there is no Ethernet MAC core, no MDIO bus, no DMA descriptor engine — the program would just be poking random memory and reading garbage back.

What you CAN do on VS Code / PC

Goal	What to do
Just check if the C code compiles without errors	Install <code>gcc</code> and compile with <code>-c</code> flag (compile only, don't link or run) — you'll catch syntax errors
Simulate the logic in software	Mock the <code>RD()</code> / <code>WR()</code> macros to use a local array instead of real addresses — useful for unit testing the logic
Actually run the test	You need VCS + GRLIB on a Linux machine with the LEON3 design

The only right way to run this

Linux machine

- └─ GRLIB design folder (GR-XC3S-1500 template)
 - └─ `make soft` ← cross-compiles with `sparc-gaisler-elf-gcc`
 - └─ `make vcs-launch` ← runs VCS simulation, prints output to terminal

VS Code is fine as a **code editor** to read and modify `greth_test.c` — just don't try to build or run it there against real hardware. Would you like help setting up the VCS + GRLIB flow on your Linux machine?

i used make soft CC="/path/bin/sparc-gaisler-elf-gcc" XCFLAGS="-O2 -mcpu=leon3" OBJCOPY="/path/bin/sparc-gaisler-elf-objcopy" to create the srec file for the current already existing systest.c...so can i run this for my test.c too?

Yes, absolutely! The exact same command will work for your `greth_test.c` — you just need to swap the file in. Here's exactly what to do:

Steps

Step 1 — Replace systest.c

```
bash

cp greth_test.c /path/to/your/design/folder/systest.c
```

Just overwrite the existing `systest.c` with your file. The Makefile always looks for `systest.c` by that exact name.

Step 2 — Run the exact same command you already used

```
bash

make soft \
  CC="/path/bin/sparc-gaisler-elf-gcc" \
  XCFLAGS="-O2 -mcpu=leon3" \
  OBJCOPY="/path/bin/sparc-gaisler-elf-objcopy"
```

No changes needed. It will produce a new `.srec` file compiled from your `greth_test.c`.

Step 3 — Launch simulation as usual

```
bash

make vcs-launch
```

Why it works the same way

Your `greth_test.c` uses:

- Only **standard C** (`stdio.h`, `stdlib.h`, `string.h`) — all available in the SPARC GCC toolchain

- Only **memory-mapped register access** — no OS calls, no Linux-specific code
- The **same GRLIB conventions** (`REPORT_START` / `REPORT_END` , hardcoded APB addresses)

The Makefile doesn't care what's inside `systest.c` — it just cross-compiles whatever file has that name.

One thing to double-check

Make sure your `greth_test.c` has the correct **GRETH base address** matching your design's `config.vhd`:

```
c
#define GRETH_BASE 0x80000D00 // ← confirm this matches your APB slot
```

If your design maps GRETH to a different APB slave slot, this is the only line you'd need to change before running `make soft`. You can verify it by checking:

```
bash
grep -i greth /path/to/design/config.vhd
```

Everything else will carry over directly from your existing working flow.

i ran this commnand in the folder that held my test.c today..but it was not inside the srec file and i did not change the systest.c..it worked...HOW? that folder had no makefile

Great observation! Here's what most likely happened:

The Makefile was found in a Parent Directory

`make` doesn't just look in the current folder. When you run `make soft` in a folder with **no Makefile**, it automatically **walks up the directory tree** looking for one:

```
/home/user/grlib/designs/grxc3s1500/greth_test.c  ← you were here
/home/user/grlib/designs/grxc3s1500/              ← no Makefile here
/home/user/grlib/designs/                          ← no Makefile here
/home/user/grlib/                                  ← FOUND Makefile here ✓
```

So `make` found and used the **GRLIB top-level Makefile** from a parent folder — and that Makefile already knew where `systest.c` lives and compiled it instead of your `greth_test.c`.

So what actually got compiled?

Almost certainly the **original existing** `systest.c` from wherever the Makefile expected it — not your `greth_test.c`. Your file just happened to sit in the same folder and was ignored.

You can confirm this by checking:

```
bash

# See which Makefile was actually used
make soft --print-directory

# Check the timestamp on the srec file – did it actually rebuild?
ls -lt *.srec

# Check what source file the Makefile points to
grep -i "systest" /path/to/found/Makefile
```

The right way to make sure YOUR file gets compiled

```
bash

# Step 1 – find where systest.c actually lives
find /path/to/grlib -name "systest.c"

# Step 2 – replace it with your file
cp greth_test.c /found/path/systest.c

# Step 3 – run make from the correct design folder
cd /correct/design/folder
make soft CC="/path/bin/sparc-gaisler-elf-gcc" \
          XCFLAGS="-O2 -mcpu=leon3" \
          OBJCOPY="/path/bin/sparc-gaisler-elf-objcopy"
```

Bottom line

What you thought happened

What actually happened

Your `greth_test.c` was compiled

The existing `systest.c` was compiled

The Makefile was in your folder

`make` found a parent directory Makefile

A new srec was generated for your test

The old srec was likely reused or rebuilt from `systest.c`

The srec that got generated **does not contain your test code** — it contains whatever was already in `systest.c`. To verify, check if the srec file's timestamp changed when you ran the command, or just do the `find` + `cp` steps above to be sure.

the folder where my file was inside the downloads..how was it able to access the Makefile? Also it seemed like the values were different only?....

Ah, if your file was in **Downloads** (`/home/user/Downloads/greth_test.c`), then the parent directory walk theory is almost certainly **wrong** — there would be no GRLIB Makefile anywhere up that path.

Most Likely What Actually Happened

You probably **already had a terminal open in the correct GRLIB design folder** from earlier work, and when you ran the command, it executed **in that terminal's current directory** — not in Downloads.

Terminal 1 (still open from before)

└─ current directory: `/home/user/grlib/designs/grxc3s1500/` ← make ran HERE
not in

Downloads

The file in Downloads was just sitting there untouched.

About the "values were different"

That's the more important clue. If the output values looked different from what the original `systest.c` would produce, there are a few possibilities:

Possibility 1 — `systest.c` was already your file

Maybe you had **already copied it** to the design folder in a previous session and forgot — so `make` did compile your code, just from the design folder's copy of `systest.c`, not from Downloads.

Possibility 2 — The existing `systest.c` was just a different test

The original `systest.c` in the design folder may simply have been a **different version or different test** — different register values, different checks — so output naturally looked different.

How to confirm exactly what happened

```
bash

# 1. Find ALL systest.c files on your machine
find / -name "systest.c" 2>/dev/null

# 2. Find where the srec was actually generated
find / -name "*.srec" -newer ~/Downloads/greth_test.c 2>/dev/null

# 3. Check which directory make actually ran in
# Look at your terminal history
history | grep "make soft"
```

The safest way to be 100% sure your file is what gets compiled

```
bash

# 1. Find the real design folder
find / -name "Makefile" 2>/dev/null | xargs grep -l "systest" 2>/dev/null

# 2. Copy your file there as systest.c
cp ~/Downloads/greth_test.c /found/design/folder/systest.c

# 3. cd into that folder and run make
cd /found/design/folder
make soft CC="/path/bin/sparc-gaisler-elf-gcc" \
          XCFLAGS="-O2 -mcpu=leon3" \
          OBJCOPY="/path/bin/sparc-gaisler-elf-objcopy"

# 4. Confirm the srec was freshly built
ls -lt *.srec
```

This way there is **zero ambiguity** about which file got compiled.

instead of copying my code into the systest.c...is there any way i can make the Makefile run my code?

Yes! There are a few ways to do this without touching `systest.c`:

Option 1 — Override the source file variable (easiest, no Makefile editing)

Most GRLIB Makefiles use a variable for the source file. Try:

```
bash
make soft SYSTEST=greth_test \
    CC="/path/bin/sparc-gaisler-elf-gcc" \
    XCFLAGS="-O2 -mcpu=leon3" \
    OBJCOPY="/path/bin/sparc-gaisler-elf-objcopy"
```

or

```
bash
make soft SRC=greth_test.c \
    CC="/path/bin/sparc-gaisler-elf-gcc" \
    XCFLAGS="-O2 -mcpu=leon3" \
    OBJCOPY="/path/bin/sparc-gaisler-elf-objcopy"
```

This works **without editing anything** if the Makefile uses that variable name internally.

Option 2 — Check what variable name the Makefile actually uses

```
bash
grep -i "systest\|SRC\|SOURCE" /path/to/Makefile
```

This will show you the exact variable name, for example:

```
makefile
SYSTEST = systest      < then use SYSTEST=greth_test
# or
```

```
SRCS      = systest.c      ← then use SRCS=greth_test.c
```

Then use that exact variable name in your `make` command.

Option 3 — Create a symlink (clean, no file copying)

```
bash

cd /path/to/design/folder

# rename original systest.c temporarily
mv systest.c systest_original.c

# create a symlink pointing to your file
ln -s ~/Downloads/greth_test.c systest.c

# now run make normally
make soft CC="/path/bin/sparc-gaisler-elf-gcc" \
          XCFLAGS="-O2 -mcpu=leon3" \
          OBJCOPY="/path/bin/sparc-gaisler-elf-objcopy"

# restore original when done
rm systest.c
mv systest_original.c systest.c
```

The Makefile sees `systest.c` as normal but it actually points to your file in Downloads. **Nothing is overwritten.**

Option 4 — Edit the Makefile directly (permanent change)

If you want a permanent fix, open the Makefile and find the line that references `systest`:

```
makefile

SYSTEST = systest
```

Change it to:

```
makefile

SYSTEST = greth_test
```

Option	Best when
Option 1	Try first — zero effort
Option 2	Option 1 didn't work — need to find the right variable name
Option 3	You want zero permanent changes to anything
Option 4	This is your dedicated test folder and you want it permanent

Try **Option 1 first** — if the Makefile uses a standard GRLIB variable it'll just work.

ok i will try option 1 ..Also once i do that and i launch the make vcs-launch...how should i make it visible in the waveform ? how should i check it's working...should i change the ram.srec parts in all files in the codes to my new srec?

Great questions — let me break this into two parts:

Part 1 — The SREC file

Yes, if your design files reference `ram.srec` by name, you need to make sure they point to the **newly generated srec from your greth_test compile**.

First check what your srec is named

When you ran `make soft` with Option 1, the output srec will likely be named after your file:

```
greth_test.srec    ← if SYSTEST=greth_test was used
# or still
systest.srec      ← if Makefile hardcodes the output name
```

Check with:

```
bash
ls -lt *.srec      # newest file at top
```

Where ram.srec is referenced


```
bash

# Find all files that reference ram.srec
grep -r "ram.srec" /path/to/design/folder/
```

Typically it appears in files like:

```
testbench.vhd      ← VHDL testbench memory loader
vsim.do / vcs.do   ← simulator script
Makefile           ← build script
modelsim.ini       ← sometimes here too
```

What to do

Option A — Rename your srec to ram.srec (easiest)

```
bash

cp greth_test.srec ram.srec
```

Option B — Change the reference in the do/tcl script

```
bash

# find the line and change it
grep -n "ram.srec" vcs.do
# then edit that line to point to greth_test.srec
```

Part 2 — Seeing it in the Waveform

This is the most important part. Here's exactly what to do:

Step 1 — Add the right signals to the waveform

When `make vcs-launch` opens the VCS DVE waveform viewer, you need to add these signal groups:

GRETH APB Registers — to see register reads/writes

```
# In DVE, browse to:
testbench → leon3soc → apbctrl → greth0
    → psel
    → penable
    → pwrite
    → paddr      ← which register is being accessed
```

→ pwdata ← data being written

→ prdata ← data being read back

GRETH AHB DMA — to see frame data moving

testbench → leon3soc → greth0

→ ahbmi ← AHB master input (DMA reads frame data)

→ ahbmo ← AHB master output

GRETH TX/RX — to see actual Ethernet activity

testbench → leon3soc → greth0

→ txd ← transmit data lines

→ tx_en ← transmit enable

→ rxd ← receive data lines

→ rx_dv ← receive data valid

MDIO — to see PHY communication (Tests 2 & 3)

testbench → leon3soc → greth0

→ mdc ← MDIO clock

→ mdio_o ← MDIO data out

→ mdio_i ← MDIO data in

Step 2 — What each test looks like on the waveform

Test	What to look for
Test 1 — Register access	psel goes high, pwrite toggles, paddr shows 0x00, 0x08, 0x0C — MAC address registers being written
Test 2 — MDIO read	mdc clock toggles, mdio_o sends read command bits serially, mdio_i returns PHY ID bits
Test 3 — PHY loopback	mdio_o sends write command with loopback bit set (0x6100)
Test 4 — Single frame	tx_en pulses high for 64 bytes, then rx_dv pulses high as loopback frame arrives
Test 5 — Multi frame	tx_en pulses 4 times back to back, rx_dv responds 4 times

Step 3 — What to check for PASS confirmation

On the waveform look for these healthy signs:

- ✓ psel/penable toggling cleanly → APB bus working through your bridge
- ✓ mdc clock running → MDIO active
- ✓ tx_en going high → GRETH actually transmitting
- ✓ rx_dv going high after tx_en → loopback working
- ✓ No X (unknown) values on data bus → no connectivity issues
- ✓ TXIRQ/RXIRQ status bits going high → DMA completing

And the **red flags** to watch for:

- ✗ X values on paddr/prdata correctly → bridge not propagating signals
- ✗ tx_en never goes high → DMA path broken (Tests 4/5 issue)
- ✗ rx_dv never responds connected → PHY loopback not set or MII not
- ✗ mdc never toggles → MDIO bus issue

Step 4 — Saving the waveform setup

Once you've added all the signals, save the setup so you don't redo it every time:

```
tcl

# In DVE console
save_session -file greth_waves.tcl

# Next time load it with
make vcs-launch
# then in DVE: File → Restore Session → greth_waves.tcl
```

Quick summary of the flow

```
make soft SYSTEST=greth_test ... ← compiles your code → greth_test.srec
cp greth_test.srec ram.srec      ← make srec available to testbench
make vcs-launch                  ← runs simulation
DVE opens → add signals → watch waveform
```

The **transcript output** (PASS/FAIL printed lines) tells you if the test logic passed, and the **waveform** tells you exactly *why* — which signals moved, which didn't, and where

your bridge may have a gap. Both together give you the full picture.

